

Sommario

1)Idee di base

GENERAZIONE CHIAVI

Algoritmo per generare chiavi RSA

CIFRARE

DECIFRARE

2)Correttezza

Dimostrazione Correttezza RSA

Inverso moltiplicativo dell'aritmetica modulare

Proprietà A dell'aritmetica modulare

Proprietà B dell'aritmetica modulare

Proprietà della Funzione Phi di Eulero:

Risultato dal Teorema di Eulero:

E SE M ED N NON FOSSERO COPRIMI?

Dimostrazione Correttezza RSA in caso di m, n non coprimi

3)Sicurezza

3)Efficienza

1)Un algoritmo efficiente per calcolare $ab \bmod q$ (per cifrare e decifrare)

2)Un algoritmo efficiente per generare p e q primi (per la generazione di chiavi)

3)Un algoritmo efficiente per calcolare l'inverso moltiplicativo di e

Algoritmo di Euclide

Dimostrazione di correttezza Algoritmo di Euclide:

Algoritmo di Euclide con invarianti

ALGORITMO DI EUCLIDE ESTESO

Algoritmo di Euclide esteso con invarianti

UTILIZZO DELL'ALGORITMO DI EUCLIDE

Algoritmo per calcolare IM (in parole)

4)Complessità

COMPLESSITÀ GENERAZIONE CHIAVI

5)Conclusioni e difficoltà

Cifrario Asimmetrico RSA

I)Quali sono le caratteristiche di RSA?

1)Idee di base

RSA è il cifrario asimmetrico più utilizzato ed è stato il primo cifrario asimmetrico noto alla comunità scientifica internazionale. In quanto cifrario asimmetrico, non prevede condivisione di chiavi tra mittente e ricevente, e per cifrare e decifrare si usano chiavi diverse. Cifratura e decifratura sono relativamente inefficienti, ed è difficile o praticamente impossibile decifrare senza avere la chiave, perché richiederebbe troppe risorse computazionali. L'idea è che **A cifra con la chiave pubblica di B: $K(B)^+$, e B decifra con la propria chiave privata, e viceversa.**

RSA è basato sulla **difficoltà di calcolare la fattorizzazione di numeri ottenuti moltiplicando numeri primi molto grandi**. Il testo in chiaro (plaintext) e il testo cifrato (ciphertext) sono numeri minori di un determinato numero n che viene detto **modulo**. Anche se si può pensare che questa sia una limitazione, in realtà non lo è per due motivi:

- con RSA ci occuperemo di cifrare quantità di testo molto piccole come delle chiavi simmetriche per far avvenire una comunicazione più efficiente, approfondiremo questo discorso più avanti.
- si può cifrare a blocchi di grandezza minore di n

Lo schema generali di RSA prevede:

- 1)Generazione chiavi
- 2)Algoritmo per cifrare
- 3)Algoritmo per decifrare

GENERAZIONE CHIAVI

Algoritmo per generare chiavi RSA

che agisce in questo modo:

- scegli p e q primo //Miller Rabin
- calcola il **modulo** $n = pq$
- scegli $e < (p-1)(q-1)$ t.c. $\text{MCD}(e, (p-1)(q-1))=1$ //Euclide esteso
- calcola $d = e^{-1} \bmod (p-1)(q-1)$ //Euclide esteso
- $K^+ = \langle e, n \rangle$, $K^- = \langle d, n \rangle$

Cifratura e decifratura sfruttano l'algoritmo efficiente per l'esponente modulare studiato con Diffie-Hellman:

CIFRARE

Avendo così definito la generazione di chiavi, possiamo dire che:

$$\begin{aligned} &\text{per cifrare } m < n: \\ &c = m^e \bmod n \end{aligned}$$

DECIFRARE

Avendo così definito la generazione di chiavi e la cifratura, possiamo dire che:

$$\begin{aligned} &\text{per decifrare } m < n: \\ &m = c^d \bmod n \end{aligned}$$

2) Correttezza

Dimostrazione Correttezza RSA

In questa dimostrazione utilizzeremo il concetto di

Inverso moltiplicativo dell'aritmetica modulare

L'inverso di un numero x in un'aritmetica modulare modulo N è quel numero y per il quale risulta
 $xy = 1 \bmod N$

la

Proprietà A dell'aritmetica modulare

$$(ab) \bmod M = (a \bmod M) * (b \bmod M) \bmod M$$

Infatti (dimostrazione):

Chiamiamo “ $a \bmod M$ ” x , e “ $b \bmod M$ ” y .

$x, y < M$ grazie all'operazione modulo.

Ne consegue che $a = M * k + x$ per un certo k , e

$b = M * j + y$ per un certo j . Perciò:

$$(a \bmod M) * (b \bmod M) \bmod M =$$

$$(M * k + x) * (M * j + y) \bmod M =$$

$$(M * k * M * j + M * k * y + M * j * x + x * y) \bmod M =$$

//tutti gli addendi a sinistra di $x*y$ sono multipli di M ,
ergo anche la loro somma è multiplo di M

$$(z*M + x*y) \bmod M =$$

//usando $(M*w + z) \bmod M = M:$

$$x*y \bmod M$$

c.v.d.

e la

Proprietà B dell'aritmetica modulare

$$(a^x) \bmod M = (a \bmod M)^x \bmod M$$

Infatti (dimostrazione):

Si tratta di un caso particolare della proprietà A:

$$(ab) \bmod M = (a \bmod M) * (b \bmod M) \bmod M //b = a$$

$$(a*a) \bmod M = (a \bmod M) * (a \bmod M) \bmod M =$$

$$a^2 \bmod M = (a \bmod M)^2 \bmod M$$

c.v.d.

Obbiettivo: dimostrare che, se $C(m) = c$, allora $D(c) = m$, ovvero che
se $c = m^e \bmod n$, allora $m = c^d \bmod n \rightarrow$ (sostituzione)

$m = (m^e \bmod n)^d \bmod n \rightarrow$ (**Proprietà B dell'aritmetica Modulare**)
 $m \equiv m^{ed} \pmod{n}$.

Poiché d è l'inverso moltiplicativo di e in modulo $(p-1)*(q-1)$,
 $e*d \equiv 1 \pmod{(p-1)*(q-1)}$.

- Dimostriamo prima che $(p-1)(q-1) = \Phi(n)$. Ricordiamo che n , p sono primi e $n = p*q$, e $\Phi(n)$ = numero di numeri coprimi con n e minori di n .

Proprietà della Funzione Phi di Eulero:

Siano p e q primi diversi tra loro; allora, $\Phi(p * q) = (p-1)*(q-1)$

Infatti (dimostrazione):

$\Phi(p * q)$ conta tutti i k tali che $0 < k < p * q$ e $\text{MCD}(k, p * q) = 1$.

Poiché p e q sono fattori primi di $p * q$, un intero k è coprimo con $p * q$ solo se non è multiplo né di p , né di q .

Nel range $0 < k < p * q$, ci sono ovviamente $(p - 1)$ possibili multipli **distinti** di q , e $(q - 1)$ possibili multipli **distinti** di p (infatti l'ultimo, $p * q$, non è compreso).

Questi due insiemi di multipli sono necessariamente disgiunti; infatti, se ci fosse un k multiplo sia di p che di q , sarebbe necessariamente $> p * q$.

Quindi, riassumendo:

- numero totale di interi da 0 a $p * q = p * q - 1$

- numero di multipli di p , da 0 a $(p * q) = q - 1$

- numero di multipli di q , da 0 a $(p * q) = p - 1$

Allora il numero di coprimi con $p * q$, da 0 a $p * q$, è:

$$\Phi(p * q) = (p * q - 1) - (p - 1) - (q - 1) =$$

$$p * q - 1 - p + 1 - q + 1 = p * q - p - q + 1 = (p - 1)(q - 1) // \text{c.v.d.}$$

Ancora più semplicemente si poteva dimostrare così:

se p e q primi, $\Phi(q) = q - 1$ e

$\Phi(p) = p - 1$ poiché ogni numero è coprimo con un primo;
poiché Φ è una funzione moltiplicativa,

$$\Phi(p * q) = \Phi(p) * \Phi(q) = (p - 1) * (q - 1) // \text{c.v.d.}$$

- Dal Teorema di Eulero ([spiegato in con Diffie-Hellman](#)), sappiamo che:

Risultato dal Teorema di Eulero:

con m, n relativamente primi, $m^{\Phi(n)} \bmod n = 1$,

quindi (**risultato utile 1**), $\forall k, m^{k * \Phi(n)} \bmod n = (m^{\Phi(n)})^k \bmod n = 1$.

Se aggiungiamo la condizione che $m < n$:

$$\forall k, m^{k * \Phi(n) + 1} \bmod n = (m^{k * \Phi(n)} * m) \bmod n =$$

// per la **Proprietà C dell'aritmetica modulare**:

$$= (m^{k * \Phi(n)}) \bmod n * m \bmod n =$$

// poiché $m < n$:

$$1 * m \bmod n = m.$$

Il **risultato utile 2** dunque è che

$$\forall k, m^{k * \Phi(n)+1} \bmod n = m.$$

Applicando questo risultato al nostro caso, dove $n = p * q$ con p e q primi, per quanto visto sopra otteniamo che

$$\forall k, m^{k * \Phi(n)+1} \bmod n = m^{k * (p-1)(q-1)+1} \bmod n = m.$$

- Sappiamo che, se $a \equiv 1 \pmod{b}$, allora $a = (k*b) + 1$, per qualche k .

Allora abbiamo che:

$$\forall k, m^{k * \Phi(n)+1} \bmod n = m^{k * (p-1)(q-1)+1} \bmod n = m$$

e poiché

$$e*d \equiv 1 \pmod{(p-1)*(q-1)},$$

allora $\exists k'$ tale che

$$e*d = k'*(p-1)*(q-1)+1$$

per qualche k , abbiamo che

per $k = k'$ (ricordiamo che $(p-1)(q-1)+1 = \Phi(n)+1$):

$$c^d \bmod n = m^{ed} \bmod n = m^{k'(p-1)(q-1)+1} \bmod n = m$$

//c.v.d.

E SE M ED N NON FOSSERO COPRIMI?

Abbiamo quindi dimostrato che $c^d \bmod n = m$, ovvero che, se il messaggio m , tradotto in un numero minore di n , ed n stesso sono relativamente primi, allora cifrando e decifrando si riottiene il messaggio originale m . Ma cosa accade se m non è relativamente primo con n , ovvero se $m = j*p$ o $m = i*q$? Funziona lo stesso, dimostriamolo nel caso $m = j*p$, il caso $m = i*q$ è analogo.

Dimostrazione Correttezza RSA in caso di m, n non coprimi

$n = p*q$

- Dato che i fattori primi di n sono p e q , questi sono i soli possibili divisori in comune tra m ed n , ovvero $\text{MCD}(n, m) > 0 \Rightarrow m$ multiplo di p oppure m multiplo di q (noi stiamo studiando il caso in cui sia multiplo di p , ma con q sarebbe lo stesso)

Tuttavia **non può esserlo di entrambi**, perché altrimenti

$$m \geq p * q \text{ ovvero } m \geq p * q \text{ //contraddice ipotesi}$$

ovvero m sarebbe maggiore o uguale a $p*q$, ovvero n ; e invece vale ancora l'ipotesi $m < n$ (RSA ammette solo messaggi $m < n$). Quindi $m = j*p$.

- Per lo stesso motivo, deve valere $j < q$, infatti se così non fosse $m = j \cdot p = q \cdot p = n$, //contraddice ipotesi
- Sappiamo che q è primo, e quindi m potrebbe avere divisori comuni con q solo se fosse multiplo di q , ma abbiamo appena escluso questa possibilità. Perciò, m e q sono **coprime**. Allora se $\text{MCD}(m, q) = 1$, si può applicare il **Teorema di Eulero**:

$$m^{\Phi(q)} \bmod q = 1$$

Per il **risultato utile 1 da Eulero**:

$$\forall x, m^{x \cdot \Phi(q)} \bmod q = 1$$

Scelgo $x = k \cdot \Phi(p)$:

$$m^{\Phi(p) \cdot \Phi(q)} \bmod q = m^{\Phi(n)} \bmod q = 1$$

Il che continua a valere per ogni esponente:

$\forall i, m^{i \cdot \Phi(n)} \bmod q = 1$, ovvero per ogni i esiste un k tale che

$$m^{i \cdot \Phi(n)} = 1 + k \cdot q.$$

moltiplico per m (ovvero $j \cdot p$) da entrambe le parti:

$$m \cdot m^{i \cdot \Phi(n)} = m + m \cdot k \cdot q$$

$$m^{i \cdot \Phi(n)+1} = m + j \cdot p \cdot k \cdot q$$

$$m^{i \cdot \Phi(n)+1} = m + j \cdot k \cdot n // p \cdot q = n$$

Ormai sappiamo che questa dicitura indica che tale numero mod n dà resto m . Otteniamo quindi

$$\forall i, m^{i \cdot \Phi(n)+1} = m^{k \cdot (p-1)(q-1)+1} \equiv m \pmod{n}$$

Sappiamo anche dalla dimostrazione precedente che $\exists k'$ tale che

$$e \cdot d = k' \cdot (p-1)(q-1)+1$$

scelgo $i = k'$, perciò

$$m^{k' \cdot (p-1)(q-1)+1} \equiv m^{e \cdot d} \equiv m \pmod{n}$$

//c.v.d.

In conclusione, se il messaggio m può essere espresso come numero minore di n - non necessariamente suo coprimo -, allora cifrando e decifrando si riottiene il messaggio originale m .

3) Sicurezza

- Si ritiene **computazionalmente difficile** il problema di decifrare un messaggio cifrato con RSA senza conoscere la corrispondente chiave d
- Si ritiene **computazionalmente difficile** calcolare d a partire da e ed n senza conoscere p e q
- Si ritiene **computazionalmente difficile** calcolare p e q a partire da n , per n grande

3) Efficienza

Gli algoritmi per cifrare e decifrare, conoscendo le corrispondenti chiavi, hanno una complessità logaritmica rispetto ad n (quindi è pari ad una complessità lineare nei bit dell'esponente). Sono relativamente inefficienti rispetto ai cifrari simmetrici. L'algoritmo per generare le chiavi, **Euclide Generalizzato**, che vedremo a breve, è relativamente inefficiente e può causare problemi per n grande o con hardware limitato. La complessità risulterà lineare nel numero di bit di n .

II) Cosa occorre per realizzare RSA?

1) Un algoritmo efficiente per calcolare $a^b \bmod q$ (per cifrare e decifrare)

Vedi la stessa sezione nel capitolo Diffie-Hellman

2) Un algoritmo efficiente per generare p e q primi (per la generazione di chiavi)

Vedi la stessa sezione nel capitolo Diffie-Hellman

3) Un algoritmo efficiente per calcolare l'inverso moltiplicativo di e

Per calcolare l'inverso moltiplicativo di e , RSA si basa sull'**Algoritmo di Euclide** per calcolare l'MCD.

Algoritmo di Euclide

che agisce in questo modo:

```
//calcolo di mcd(a,b) (con  $a \geq b$ , altrimenti invertiamo)
Z = a; W = b; //mcd(W,Z) = mcd(Z,W) = mcd(a,b)
while (TRUE) {
  if (W == 0) return Z; //Z = mcd(a,b)
  R = Z mod W;
  Z = W; W = R;}

```

Esempio: mcd(99,81)

i	a	b	Z	W	R	
0	99	81	99	81	18	$(99=1*81+18=81+18)$
1	99	81	81	18	9	$(81=4*18+9=72+9)$
2	99	81	18	9	0	$(18=2*9+0=18+0)$
3	99	81	9	0		MCD(99,81)=9

i	a=r ₀	b=r ₁	
0	$a=r_0=q_1*b+r_2=q_1*r_1+r_2$		$(99=1*81+18=81+18)$
1	$b=r_1=q_2*r_2+r_3$		$(81=4*18+9=72+9)$
2	$r_2=q_3*r_3+r_4=q_3*r_3+0$		$(18=2*9+0=18+0)$
3	$r_3=\text{MCD}(a,b)=\text{MCD}(r_0,r_1)$		MCD(99,81)=9

In generale: $r_k = q_{k+1} * r_{k+1} + r_{k+2}$

Dimostrazione di correttezza Algoritmo di Euclide:

1) Per ipotesi sia d tale che **$d|a$ e $d|b$** . Vogliamo dimostrare che, allora, **$d|r_i$ per ogni i** . Sappiamo che: $r_k = q_{k+1} * r_{k+1} + r_{k+2}$.

Caso base: $a=r_0$, e $b=r_1$, quindi $d|r_0$ e $d|r_1$.

$$r_0 = r_1 * q + r_2$$

ovvero esistono m, n tali che

$$m*d = n*d*q + r_2$$

$$\begin{aligned}\Rightarrow r_2 &= m*d - n*d*q \\ \Rightarrow r_2 &= d*(m - n*q) \\ //ovvero r_2 \text{ è multiplo di } d, \text{ cioè } d|r_2\end{aligned}$$

Passo induttivo

Dimostriamo che se vale per r_k e r_{k+1} allora vale per r_{k+2} qualsiasi.
Infatti se $r_k|d$ e $r_{k+1}|d$, allora:

$$\begin{aligned}r_k &= q_{k+1} * r_{k+1} + r_{k+2} \\ m*d &= q_{k+1}*n*d + r_{k+2} \\ r_{k+2} &= m*d - q_{k+1}*n*d \\ r_{k+2} &= d(m - q_{k+1}*n) \\ //ovvero r_{k+2} \text{ è multiplo di } d, \text{ ovvero } d|r_{k+2}\end{aligned}$$

A questo punto possiamo vedere che il caso base copre r_0, r_1 e r_2 , e il passo induttivo copre r_k per ogni $k-2 > 0$, quindi essi coprono tutti i possibili indici di resto, c.v.d.

2)Immaginiamo di essere giunti all'**ultimo step dell'algoritmo**, quando cioè $r_{k+2} = 0$. Vogliamo dimostrare che con k corrispondente all'ultimo step, vale che **$r_{k+1}|r_i$ per ogni i** .

Caso base

$$r_k = q_{k+1} * r_{k+1} + 0 \Rightarrow r_{k+1}|r_k$$

e sappiamo ovviamente che $r_{k+1}|r_{k+1}$.

Passo induttivo

Supponiamo che se $r_{k+1}|r_j$ e $r_{k+1}|r_{j+1}$, allora $r_{k+1}|r_{j-1}$ (siamo all'ultimo step quindi facciamo induzione "dall'alto verso il basso"):
per qualche m, n $r_{k+1}*m = r_j$ e $r_{k+1}*n = r_{j+1}$;

$$\begin{aligned}r_{j-1} &= q_j * r_j + r_{j+1} \\ r_{j-1} &= q_j * m*r_{k+1} + n*r_{k+1} \\ r_{j-1} &= r_{k+1} * (q_j * m + n) \\ //ovvero r_{j-1} \text{ è multiplo di } r_{k+1}, \text{ cioè } r_{k+1}|r_{j-1}\end{aligned}$$

A questo punto possiamo vedere che il caso base copre gli ultimi due resti, e il passo induttivo tutti quelli precedenti, perciò $r_{k+1}|r_i$ per ogni i , c.v.d.

3) Mettiamo insieme le conclusioni dei punti 1) e 2).

da 2) so che $r_{k+1}|r_i$ per ogni i (con k ultimo step dell'algoritmo), quindi anche $r_0 = a$ e $r_1 = b$:

$r_{k+1}|r_0$ e $r_{k+1}|r_1$ ovvero $r_{k+1}|a$ e $r_{k+1}|b$, ovvero

r_{k+1} è un divisore di a e b .

da 1) so che $d|a$ e $d|b$, allora $d|r_i$ per ogni i , quindi anche per r_{k+1} :

$d|r_{k+1}$ quindi $d \leq r_{k+1}$

Concludiamo quindi che r_{k+1} , il risultato dell'algoritmo è senz'altro un divisore di a e b ; e un **qualsiasi altro divisore d di a e b che possiamo scegliere, sarà minore o uguale di r_{k+1}** . Ovvero **r_{k+1} è M.C.D.(a , b), C.V.D.**

Algoritmo di Euclide con invarianti

```
//calcolo di mcd(a,b) (con  $a \geq b$ , altrimenti invertiamo)
Z=a; W=b; i=0 // $a=r_0$ ,  $b=r_1$ ,  $Z=r_i$ ,  $W=r_{i+1}$ 
while (TRUE) { // $a=r_0$ ,  $b=r_1$ ,  $Z=r_i$ ,  $W=r_{i+1}$ 
    if (W==0) return Z; //last step:  $Z=r_i=\text{mcd}(a,b)$ ,  $i=k+1$ ;  $W=r_{k+2}=0$ 
    Q = Z div W; R = Z mod W; // $Q=q_{i+1}$ ,  $R=r_{i+2}$ 
    Z = W; W = R; // $Q=q_{i+1}$ ,  $Z=r_{i+1}$ ,  $W=R=r_{i+2}$ 
    i = i+1; // $Q=q_i$ ,  $Z=r_i$ ,  $W=R=r_{i+1}$ 
}
```

ALGORITMO DI EUCLIDE ESTESO

Usiamo l' **Identità di Bezout**: se m e n interi non nulli e d è il loro MCD, $\exists (a,b)$ tali che $am + bn = d$, cioè d può essere scritto come combinazione lineare. Abbiamo già dimostrato che i resti dell'algoritmo possono scriversi come combinazioni lineari di a e b per coefficienti interi x e y . Ora estendiamo Euclide per trovare, oltre a $\text{MCD}(a, b)$ anche i coefficienti x e y che combinati con a e b danno l'MCD.

Algoritmo di Euclide esteso

```
//calcolo di  $r_{k+1}=\text{mcd}(a,b)$  e  $x,y$  t.c.  $ax+by=\text{mcd}(a,b)$ 
Z=a; W=b; i=0,  $X_2=1$ ,  $Y_2=0$ 
Z=b; W=a mod b; Q=a div b; i=1;  $X_1=0$ ;  $Y_1=1$ ;
while (T) {
    X= $X_1$ ; Y= $Y_1$ ;
    if (W == 0) return Z,X,Y;
    X= $X_2-Q*X_1$ ; Y= $Y_2-Q*Y_1$ ;
    Q = Z div W; R = Z mod W; Z = W; W = R;
     $X_2,Y_2X_1,Y_1$ ;  $X_1,Y_1X,Y$ ; i=i+1;
}
```

Algoritmo di Euclide esteso con invarianti

```
//calcolo di  $r_{k+1}=\text{mcd}(a,b)$  e  $x,y$  t.c.  $ax+by=\text{mcd}(a,b)$ 
Z=a; W=b; i=0,  $X_2=1$ ,  $Y_2=0$ 
//i=0,  $a=r_0$ ,  $b=r_1$ ,  $Z=r_i$ ,  $W=r_{i+1}$ ,  $X_2=x_i$ ,  $Y_2=y_i$ ,  $ax_i+by_i=r_i$ 
Z=b; W=a mod b; Q=a div b; i=1;  $X_1=0$ ;  $Y_1=1$ ;
//i=1,  $Z=r_i$ ,  $W=r_{i+1}$ ,  $Q=q_i$ ,  $X_1=x_i$ ,  $Y_1=y_i$ ,  $X_2=x_{i-1}$ ,  $Y_2=y_{i-1}$ 
// $ax_i+by_i=r_i$ ,  $ax_{i-1}+by_{i-1}=r_{i-1}$ 
while (T) {
    //Z= $r_i$ , W= $r_{i+1}$ , Q= $q_i$ ,  $X_1=x_i$ ,  $Y_1=y_i$ ,  $X_2=x_{i-1}$ ,  $Y_2=y_{i-1}$ 
    X= $X_1$ ; Y= $Y_1$ ; //X= $x_i$ , Y= $y_i$ 
    if (W == 0) return Z,X,Y; //Z= $r_i=\text{mcd}(a,b)=aX+bY$ 
    X= $X_2-Q*X_1$ ; //X= $x_{i-1}-q_i*x_i=x_{i+1}$ 
    Y= $Y_2-Q*Y_1$ ; //Y= $y_{i-1}-q_i*y_i=y_{i+1}$ 
    Q = Z div W; R = Z mod W; //Q =  $q_{i+1}$ , R= $r_{i+2}$ 
    Z = W; W = R; //Q= $q_{i+1}$ , Z= $r_{i+1}$ , W=R=  $r_{i+2}$ 
     $X_2,Y_2X_1,Y_1$ ;  $X_1,Y_1X,Y$ ; //X= $x_i$ , Y= $y_i$ , X= $x_{i+1}$ ; Y= $y_{i+1}$ 
    i=i+1; // Q= $q_i$ , Z= $r_i$ , W=R= $r_{i+1}$ , X= $x_{i-1}$ , Y= $y_{i-1}$ , X= $x_i$ , Y= $y_i$ 
}
```

UTILIZZO DELL'ALGORITMO DI EUCLIDE

Come si usa l'**Algoritmo di Euclide** per calcolare l'inverso moltiplicativo di un numero?

Algoritmo per calcolare IM (in parole)

- 1) Dati a, b tali che $\text{MCD}(a, b) = 1$, voglio calcolare t tale che $b \cdot t \bmod a = 1$.
- 2) Usando **Euclide Generalizzato**, ottengo X e Y tali che $(X \cdot a + Y \cdot b) = \text{MCD}(a, b) = 1$
- 3) Poiché $X \cdot a + Y \cdot b = 1$, $Y \cdot a = 1 - X \cdot a$, perciò $Y \cdot b \bmod a = 1$.

4) Complessità

COMPLESSITÀ GENERAZIONE CHIAVI

Richiamiamo l'algoritmo iniziale:

Algoritmo per generare chiavi RSA

che agisce in questo modo:

- 1) scegli p e q primo // *Miller Rabin*
- 2) calcola il **modulo** $n = pq$
- 3) scegli $e < (p-1)(q-1)$ t.c. $\text{MCD}(e, (p-1)(q-1)) = 1$ // *Euclide esteso*
- 4) calcola $d = e^{-1} \bmod (p-1)(q-1)$ // *Euclide esteso*
- 5) $K^+ = \langle e, n \rangle$, $K^- = \langle d, n \rangle$

Per i punti 3) e 4) occorre Euclide Esteso, quindi la complessità nel caso medio è logaritmica in n (perché il resto di x/y è in media $y/2$).

5) Conclusioni e difficoltà

- Moltiplicare numeri molto lunghi (con tanti bit) su una architettura a 64 bit può essere essa stessa una operazione dispendiosa. In particolare se sfiorano la lunghezza dei registri occorre fare la moltiplicazione “in colonna”. Tuttavia resta una complessità quadratica, dunque polinomiale.
- Affinché RSA funzioni, un possibile attaccante non deve essere in grado, vedendo n , di trovare p e q , ovvero non deve poter fare la fattorizzazione di n . Per ottenere questo, il modulo deve essere sufficientemente grande. Quanto grande? La risposta a tale domanda si ottiene sperimentalmente; per applicazioni civili si usano moduli di circa 1024 bit, per applicazioni governative o militari anche 2048.
- Cosa succede se ho un messaggio m più lungo del modulo n ? Dovrei spezzarlo in blocchi e usare RSA come cifrario a blocchi, ma normalmente non si fa; si usa tendenzialmente RSA per cifrare una chiave simmetrica, e poi uso un cifrario simmetrico per cifrare il resto del testo.